Support-Conditioned LoRA for Test-Time Speaker Adaptation in Visual Speech Recognition

Cashel Fitzgerald
Cornell Tech
cf566@cornell.edu

Jihyun Kim
Cornell Tech
jk3257@cornell.edu

Abstract

Visual Speech Recognition (VSR) systems achieve strong performance on curated benchmarks but degrade under domain shift, particularly when encountering unseen speakers or recording conditions. We study a parameter-efficient adaptation method for VSR based on support-conditioned Low-Rank Adaptation (LoRA) produced in test-time, a pretrained Auto-AVSR model.

Our approach inserts LoRA modules into selected encoder layers and uses a hypernetwork to predict their parameters from a short video support prefix (the first few frames of each input for instance). This enables gradient-free personalization at test time: the model adapts to a new speaker using only unlabeled prefix frames, without fine-tuning or additional optimization.

On the LRS2 dataset, our method yields a 4.1% relative WER reduction (51.5% $\longrightarrow$ 49.4%) in unseen data over an unadapted Auto-AVSR baseline using a $\sim 2.56 \times 10^7$-parameter hypernetwork. Our findings suggest that support-conditioned adapters are a promising direction for efficient test-time adaptation in VSR, but practical gains depend sensitively on model and conditioning design.

1 Introduction

Visual Speech Recognition (VSR), or lip reading, aims to transcribe spoken words from video alone by analyzing lip motion. VSR has a few inherent challenges compared to audio-based speech recognition. For example, many phonemes share similar visual patterns (visemes), making clear distinction difficult. Visual input around lips has lower temporal resolution, and small variations in pose, lighting, or camera angle can also significantly impact appearance.

Recent work in self-supervised and audio-visual models have achieved impressive results on curated benchmarks. AV-HuBERT [1] uses over 1,700 hours of multimodal pretraining, while Auto-AVSR [2] uses pseudo-labeling to improve data use efficiency. In a complementary direction, VALLR [3] demonstrates that highly performant visual-only performance can be achieved with supervised learning on only ~30 hours of labeled data through a phoneme-centric two-stage pipeline and a pretrained LLM in the phoneme to sentence stage. However, these models remain sensitive to domain shift—performance drops notably when encountering unseen speakers or recording conditions.

To address this limitation, we propose *Support-Conditioned LoRA for Test-Time Speaker Adaptation in Visual Speech Recognition*, a new adaptation approach that personalizes a frozen pretrained model using only the initial frames of each video. Our method freezes the Auto-AVSR visual backbone and adds trainable Low-Rank Adaptation (LoRA) modules into encoder layers. A hypernetwork is used to predict LoRA weights based on encoded features from the first few frames (prefix), enabling the model to adapt to speaker specific articulation patterns, head poses, and visual characteristics for test-time.

Our primary contributions are:

- We formulate **Support-Conditioned LoRA for VSR** using a hypernetwork that predicts LoRA parameters from initial video frames.
- We implement this approach in PyTorch Lightning with the pretrained **Auto-AVSR** backbone, demonstrating parameter-efficient adaptation without speaker labels.
- We provide detailed implementation analysis including model experiments, architecture choices, training strategies, and the gradient-free inference process.

2 Related Work

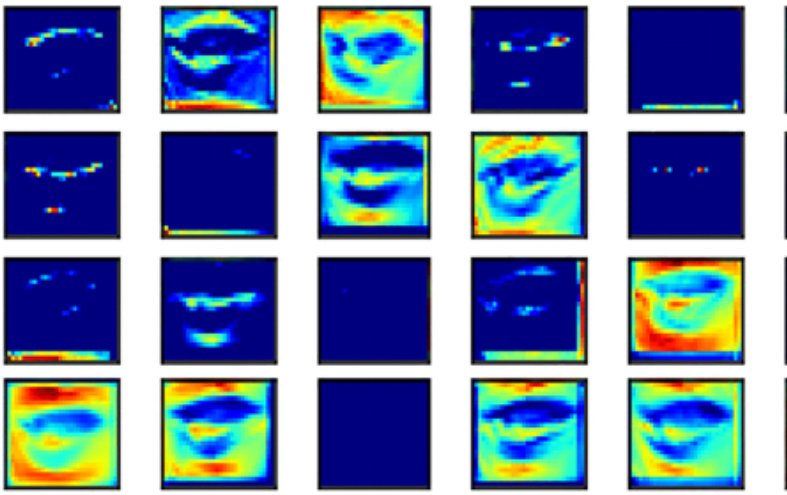

Figure 1: Lip reading input images.

2.1 Visual Speech Recognition

Visual Speech Recognition has unique challenges due to viseme ambiguity and limited visual bandwidth. Recent breakthroughs leverage large scale pretraining: AV-HuBERT [1] learns robust representations through masked multimodal prediction, while Auto-AVSR [2] generates pseudo-labels for large-scale supervised refinement.

VALLR [3] demonstrates that competitive VSR can be achieved with 30 hours of labeled data using a two-stage phoneme-centric pipeline. Despite these advances, robustness in domain shift is still challenging.

2.2 Parameter-Efficient Adaptation

Low-Rank Adaptation (LoRA) [4] enables efficient fine-tuning by freezing base models and learning small low-rank matrices. Extensions on LoRA like QLoRA [5] and AdaLoRA [6] have further improved efficiency. Our work focuses on predicting LoRA parameters efficiently at inference time rather than re-learning the whole matrix of weights.

2.3 Hypernetworks and Dynamic Adaptation

Hypernetworks [7] generate model weights from context input, enabling conditional adaptation. In VSR, dynamic adaptation has been explored through attention mechanisms and speaker feature modulation, but hypernetwork-based LoRA prediction for video prefixes is less explored.

3 Problem Statement and Data

3.1 Formal Problem Definition

We consider a VSR model f that takes a video sequence $V \in \mathbb{R}^{T\times H\times W\times 3}$ of T frames and outputs a corresponding text transcript. Our objective is to train the model to adapt to speaker specific or environment specific characteristics using only the first P frames of the video (the *prefix*).

Given:

- A frozen pretrained encoder f_{enc}
- A video prefix $V_{1:P}$ (first P frames)
- The full video $V_{1:T}$

We aim to:

(1) Extract conditioning features $c = g(f_{\text{enc}}(V_{1:P}))$ from the prefix
(2) Generate LoRA adapter weights $\{\Delta W_l\} = h(c)$ via hypernetwork h
(3) Apply these adapters to decode the full sequence with better accuracy

This approach enables per-video personalization without requiring:

- Speaker identity labels
- Multiple support clips from the same speaker
- Gradient updates at inference time

3.2 Dataset

We perform all experiments on the **LRS2** dataset, a widely used benchmark for VSR containing BBC footage clips with diverse British speakers, speaking styles, and real-world camera conditions.

3.2.1 Dataset Statistics. LRS2 consists of:

- **Training set**: Thousands of utterances from diverse speakers
- **Validation set**: Held-out speakers and conditions
- **Test set**: Additional held-out data for final evaluation

The dataset includes natural variations in:

- Speaker articulation and lip geometry
- Head pose and camera angles
- Lighting conditions
- Background clutter and occlusions

3.2.2 Preprocessing Pipeline. Following Auto-AVSR preprocessing:

(1) Resample videos to 25 fps
(2) Detect faces and extract mouth-region crops
(3) Resize to model input resolution (224×224)
(4) Normalize according to pretrained backbone requirements
(5) Apply temporal segmentation for consistent sequence lengths

3.3 Evaluation Metrics

We evaluate using:

- **Word Error Rate (WER):** Primary metric at word level
- **Character Error Rate (CER):** Secondary metric at character level

4 Method

4.1 Auto-AVSR Visual Backbone

Auto-AVSR combines a 3D convolutional front-end with a Conformer encoder for visual speech representation learning. Given a video V, the encoder produces:

$$Z = f_{\text{enc}}(V) \in \mathbb{R}^{T'\times D} \tag{1}$$

where T' is the downsampled temporal dimension and D is the feature dimension.

A CTC head maps Z to per-frame character distributions:

$$\hat{Y} = \text{softmax}(W_{\text{ctc}}Z + b_{\text{ctc}}) \tag{2}$$

During training, we freeze all pretrained weights of this backbone.

4.2 Low-Rank Adaptation (LoRA)

LoRA adds trainable low-rank matrices into linear layers. For a weight matrix $W \in \mathbb{R}^{D\times D}$:

$$W' = W + \Delta W, \quad \Delta W = BA \tag{3}$$

where $A \in \mathbb{R}^{r\times D}$, $B \in \mathbb{R}^{D\times r}$, and $r \ll D$.

The low-rank update is scaled by α/r:

$$y = Wx + \frac{\alpha}{r}(BA)x \tag{4}$$

4.3 HyperLoRA Linear Layer

We implement a custom `HyperLoRALinear` layer that accepts dynamically generated LoRA weights per batch:

4.4 LoRA Integration and Hypernetwork Conditioning

We integrate LoRA modules into selected 3D convolution and Conformer layers of the Auto-AVSR visual backbone. For a weight matrix $W_l \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ in layer l, we define the adapted weight

$$W_l' = W_l + \frac{\alpha}{r}B_lA_l, \tag{5}$$

where $A_l \in \mathbb{R}^{r\times d_{\text{in}}}$ and $B_l \in \mathbb{R}^{d_{\text{out}}\times r}$ are low-rank matrices. We follow Hu et al. and scale the low-rank update by α/r, where α is a global hyperparameter.

During training, W_l remains frozen while A_l, B_l, and the hypernetwork parameters ϕ are updated. Given a prefix $V_{1:P}$, we compute

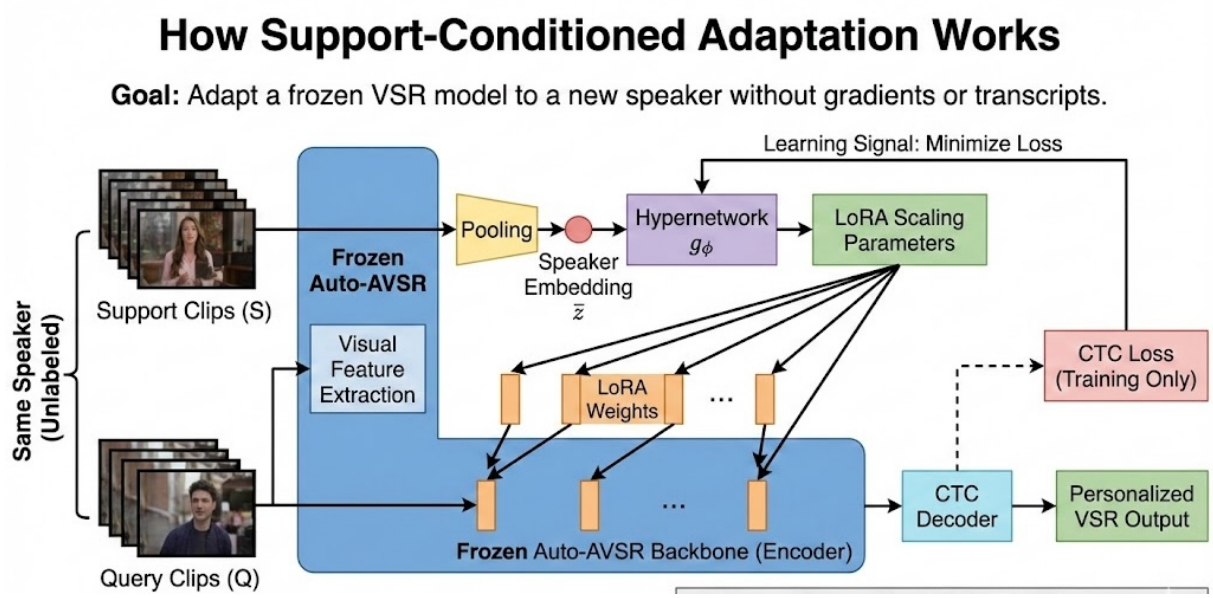

Figure 2: This shows how support conditioning works

its encoded representation

$$\bar{z} = \mathrm{MeanPool}\big(f_{\mathrm{enc}}(V_{1:P})\big) \tag{6}$$

and feed it to an MLP hypernetwork h_ϕ that outputs the full set of low-rank adapter weights for all adapted layers:

$$\{\mathrm{vec}(A_l), \mathrm{vec}(B_l)\}_{l\in\mathcal{L}} = h_\phi(\bar{z}), \tag{7}$$

where $\mathcal{L}$ is the set of adapted layers and $A_l \in \mathbb{R}^{r\times d_{\mathrm{in},l}}$, $B_l \in \mathbb{R}^{d_{\mathrm{out},l}\times r}$ are the LoRA matrices for layer l. At inference time, we perform a single forward pass on the prefix to obtain $\{A_l, B_l\}_{l\in\mathcal{L}}$, instantiate all W_l', and then decode the full sequence with the adapted backbone. No gradients are computed at test time.

Key features are:

- Base weights remain frozen
- LoRA matrices are set via `set_adapter()` before forward pass
- Supports batched adapters (different LoRA weights per sample)
- Uses einsum for efficient batched matrix operations

4.5 Encoder Conditioning Module

The feature conditioning encoder extracts a fixed size representation from the encoded prefix $\bar{z}$.

$$c = \mathrm{MLP}(\bar{z}) \tag{8}$$

We use a two-layer MLP with ReLU activations. The mask ensures padding frames don't contribute to the conditioning vector.

4.6 LoRA Hypernetwork

The hypernetwork generates all LoRA adapter weights from the conditioning vector. This is the implementation:

```
class LoRAHypernetwork(nn.Module):
    def __init__(self, specs, rank, hidden_size):
        # Calculate total parameters needed
        total_params = sum(
            (spec.in_features + spec.out_features)
            * rank
            for spec in specs
        )

        self.generator = nn.Sequential(
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, total_params),
        )

    def forward(self, conditioning):
        weights = self.generator(conditioning)
        # Parse into individual A and B matrices
        adapters = {}
        idx = 0
        for spec in self.specs:
            a_size = spec.rank * spec.in_features
            b_size = spec.out_features * spec.rank

            lora_A = weights[:, idx:idx+a_size]
            .view(
                -1, spec.rank, spec.in_features
            )
            idx += a_size

            lora_B = weights[:, idx:idx+b_size].view(
                -1, spec.out_features, spec.rank
            )
```

```
        idx += b_size

        adapters[spec.name] = (lora_A, lora_B)
    return adapters
```

4.7 Adapter Placement Strategy

We selectively add LoRA adapters based on these criteria targeting:

- Conformer encoder multi-head attention K and V projections

For this project, we are not adapting:

- Frontend 3D convolutions which are too early in the pipeline
- Positional encoding (architectural, not content-specific)
- CTC output head (task-specific)
- Conformer feedforward layers (too many parameters)
- Convormer encoder multi-head attention Q projections (saves parameter count, works well enough just augmenting k and v)

4.8 Training Procedure

Key aspects of our training procedure are:

- Prefix encoding will not have gradient flow through frozen backbone
- Each video gets its own set of LoRA adapters
- Full sequence encoding uses the dynamically set adapters
- Only hypernetwork and conditioning encoder receive gradients

The training process follows these steps:

Algorithm 1 Support-Conditioned Hypernetwork Training

0: Initialize pretrained encoder f_{enc}
0: Freeze all base weights
0: Initialize conditioning encoder g and hypernetwork h
0: **while** not converged **do**
0: Sample batch of videos $\{V^{(i)}\}$
0: **for** each video $V^{(i)}$ **do**
0: *// Extract prefix features*
0: $V_{\text{prefix}} \leftarrow V^{(i)}_{1:P}$
0: $z \leftarrow f_{\text{enc}}(V_{\text{prefix}})$ {No gradients}
0: $c^{(i)} \leftarrow g(z)$
0: *// Generate adapters via hypernetwork*
0: $\{A^{(i)}_l, B^{(i)}_l\} \leftarrow h(c^{(i)})$
0: **for** each layer l **do**
0: $\texttt{layer}_l\texttt{.set_adapter}(A^{(i)}_l, B^{(i)}_l)$
0: **end for**
0: **end for**
0: *// Forward pass with adapted model*
0: Encode full sequences with applied adapters
0: Compute CTC loss + attention loss
0: Backpropagate through g and h only
0: Update parameters with AdamW
0: **end while**=0

4.9 Inference Process

What we do at inference time is:

- Single forward pass through prefix encoder (frozen)
- Single forward pass through conditioning encoder
- Single forward pass through hypernetwork
- Single forward pass through full encoder (with adapters)
- **No gradient computation or optimization steps**

5 Experiments

5.1 Implementation Details

Table 1: Model configuration and hyperparameters.

Component	Configuration
Backbone	
Pretrained model	Auto-AVSR base
Encoder dimension	512
Modality	Video
LoRA	
Rank r	16
Alpha α	32.0
Scaling	$\alpha/r = 2.0$
Dropout	0.05
Target layers	Encoder K/V
Conditioning	
Prefix frames P	16
Conditioning dim	256
Pooling	Masked mean
HyperNetwork	
Hidden dim	256
Layers	2-layer MLP (ReLU)
Output dim	$\approx 9.8 \times 10^3$ (LoRA params)
Total params	$\approx 2.56 \times 10^7$
Training	
Optimizer	AdamW
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Learning rate warmup	3 epochs linear
Scheduler	Cosine annealing
Max epochs	20
Batch strategy	Length-aware
Max frames/batch	1600

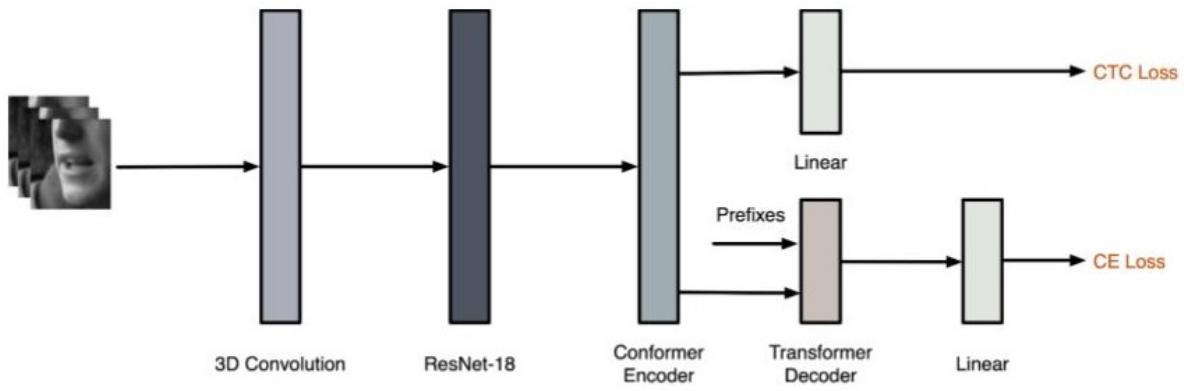

Figure 3: This shows the Auto-AVSR baseline architecture. In our method, the prefix features are encoded and used to condition the hypernetwork that generates adapter weights.

5.1.1 Training Strategy. We use PyTorch Lightning with:

- Length-aware bucketing to minimize padding
- Dynamic batch sizing based on frame count
- Model checkpointing on validation CER
- Learning rate monitoring

The loss combines:

$$\mathcal{L} = \mathcal{L}_{\text{CTC}} + \lambda \mathcal{L}_{\text{att}} \tag{9}$$

where the CTC loss ensures alignment. The attention loss, if available, provides auxiliary supervision.

5.2 Baseline Comparisons

We compare against:

Frozen Backbone: Pretrained Auto-AVSR without any adaptation. This represents the zero-shot performance on our data.

Lora on LRS2: Lora in the same configuration as the hypernetwork generated, trained on LRS2

Support-Conditioned LoRA (Ours): Adapter generation from prefix frames with single-pass inference.

5.3 Evaluation Protocol

For each method, we measure:

- **CER** (Character Error Rate): Fine-grained transcription accuracy
- **WER** (Word Error Rate): Word-level accuracy

We evaluate on the LRS2 validation set, encompassing:

- Different speaker characteristics
- Speaker poses
- Different lighting conditions

5.4 Main Result

Table 2 reports WER on the LRS2 validation set. Our support-conditioned LoRA model improves over the frozen Auto-AVSR baseline by 4.1% relative WER.

Table 2: Recognition performance on LRS2 validation set.

Method	WER (%)	Improvement
Auto-AVSR (frozen)	51.5	–
Classic LoRA	50.9	1.2%
Ours (support-conditioned LoRA)	**49**.4	**4.1%**

5.5 Failure Mode Analysis

We noticed some common failures:

- **Insufficient prefix information**: When prefix frames don't capture enough speaker characteristics (e.g., subject not yet speaking)
- **Rapid visual changes**: When video characteristics change dramatically after the prefix (e.g., sudden camera angle shift)
- **CTC alignment issues**: Inherent CTC limitations persist (blank collapse, phoneme ambiguity)
- **Prefix vs. full sequence mismatch**: When prefix is unrepresentative of the full sequence

6 Discussion and Lessons Learned

6.1 Prefix-Based Conditioning

Although our original proposal focused on few-shot support sets from multiple videos per speaker, after experiments, our implementation focused on conditioning on a short prefix from the same video. This design offered practical advantages, as it didn't require

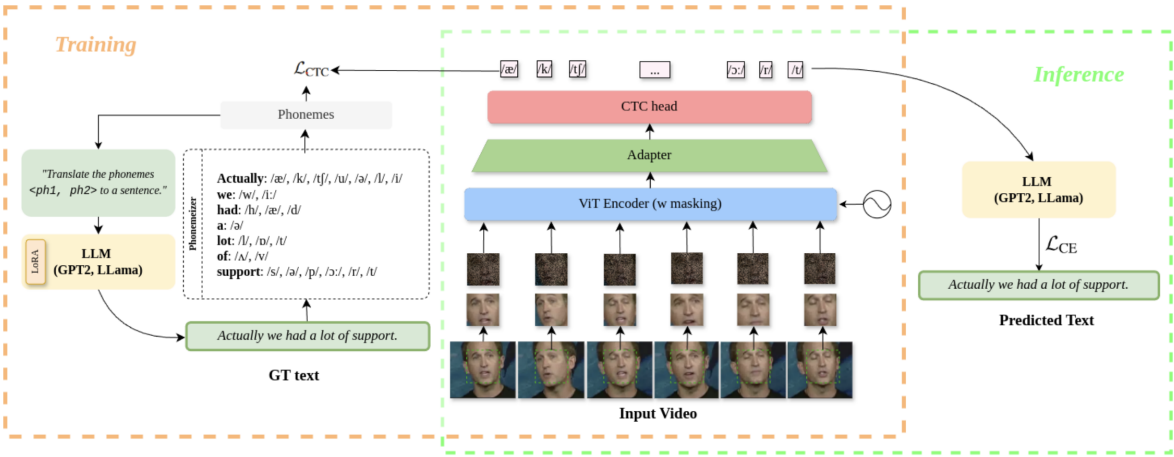

Figure 4: An overview of VALLR approach. First they extract facial regions from 16 frames of input video then apply random pixel masking, add positional embedding, and encode visual features via a vision transformer encoder. They implement temporal downsampling and use a CTC linear head to predict sequences of phonemes. During training, they also fine-tune an LLM to reconstruct sentences from phonemes with a text-only dataset.

speaker identity or separate support clips and it works on isolated videos at test time. However, the prefix contains limited information about speaker and environment, especially when restricted to only a few frames, which constrains the amount of adaptation the hypernetwork can provide.

6.2 Hypernetwork Capacity and Trade-Offs

Our experiments indicate that to get a meaningful gains from support-conditioned LoRA, we require a non-trivial sized hypernetwork. Smaller MLPs with only a few million parameters produced negligible WER improvements, whereas the larger parameter variant achieved a 4.1% relative WER reduction over the frozen backbone. This highlights a fundamental trade-off: gradient-free adaptation is feasible and computationally cheap at inference, but it shifts complexity into the hypernetwork and increases overall model size.

6.3 Negative Results from VALLR-Style Training

In our original proposal, we wanted to train a VALLR-style [3] Video Transformer with CTC head from scratch before adopting Auto-AVSR as our frozen backbone. Despite a few design experiments, these efforts did not work mainly due to *CTC blank collapse*—the model converged to predicting mostly blank tokens, yielding empty transcriptions. Through systematic debugging, we identified two fundamental issues:

6.3.1 CTC Alignment Failure and Mode Collapse. Visual speech exhibits minimal motion (2-5 pixels per frame), resulting in severe viseme ambiguity where phonemes like /p/, /b/, /m/ appear identical. CTC theoretically only requires $T' \geq |Y|$ frames, but in practice, stable alignments are obtained with $T' \geq 2 \times |Y|$ [10]. Aggressive temporal downsampling (stride-4/8) can violate this constraint. Once alignment fails, the CTC loss $\mathcal{L}_{\text{CTC}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(Y)} P(\pi|X)$ degenerates: the blank token becomes the default safe local minimum. When the model gives a very high probability to the blank symbol, gradients for non-blank tokens become very small, making it difficult to correct errors. In our experiments, validation loss plateaued at ~3-4 with almost 0 accuracy, producing completely empty transcriptions.

6.3.2 Lessons and Solutions. This analysis suggests several mitigation strategies: (1) aggressive mouth cropping to increase relative motion, (2) curriculum learning starting with short, high-motion utterances, (3) explicit blank token penalties, (4) conservative temporal downsampling, and (5) motion-aware backbones like AV-HuBERT [1] or VideoMAE [12] pretrained on temporal objectives. These failures motivated our pivot to pretrained audio-visual backbones where stable, phonetically-structured representations enable successful hypernetwork training.

7 Conclusion

We investigated support-conditioned LoRA for gradient-free VSR adaptation. By freezing Auto-AVSR and using a hypernetwork to predict LoRA weights from video prefixes, we achieved 4.1% relative WER reduction on LRS2 without labels or test-time gradients. Our key finding is that meaningful adaptation requires substantial hypernetwork capacity, revealing an information bottleneck from short prefixes: 4-16 consecutive frames provide limited speaker information, necessitating rich models to extract useful adaptation signals.

Our contributions include: (1) first investigation of support-conditioned hypernetwork adaptation for VSR, (2) working implementation achieving meaningful gains, (3) analysis of capacity requirements and information bottlenecks, (4) detailed examination of failed approaches (training from scratch), and (5) practical guidance on trade-offs versus fine-tuning.

This approach showed a clear trade-off for specific scenario: best for high-throughput scenarios processing diverse speakers where adaptation speed matters and storage is abundant, but not ideal for resource-constrained deployment or when larger improvements are required. Future work should explore longer prefixes or support sets, hierarchical architectures for efficiency, and hybrid methods combining hypernetwork prediction with lightweight gradient refinement.

References

[1] B. Shi, W.-N. Hsu, K. Lakhotia, and A. Mohamed. Learning audio-visual speech representation by masked multimodal cluster prediction. In *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[2] P. Ma, S. Petridis, and M. Pantic. Auto-AVSR: Audio-Visual Speech Recognition with Automatic Labels. In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[3] M. Thomas, E. Fish, and R. Bowden. VALLR: Visual ASR Language Model for Lip Reading. In *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2025.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[6] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, and T. Zhao. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. In *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[7] D. Ha, A. Dai, and Q. V. Le. Hypernetworks. In *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[8] C. Finn, P. Abbeel, and S. Levine. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *Proc. International Conference on Machine Learning (ICML)*, 2017.

[9] J. Snell, K. Swersky, and R. Zemel. Prototypical Networks for Few-shot Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. In *Proc. International Conference on Machine Learning (ICML)*, 2006.

[11] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *Proc. International Conference on Learning Representations (ICLR)*, 2021.

[12] Z. Tong, Y. Song, J. Wang, and L. Wang. VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.